

Reinforcement Learning for Risk Averse Selection Using Supervised Learning Simulated Environment - Baseball Pitch Selection

Patrick Hartnett
Stanford University
phartnet@stanford.edu

1 Introduction

Many simple reinforcement learning reward functions implicitly maximize the expected value of its inputs. For example in portfolio optimization this results in selecting the portfolio with the highest returns, in healthcare the policy that yields the best prognosis, and in navigation the route that's quickest all on average. However, portfolio managers would trade a few dollars for protection from a big drawdown, doctors a few days on the prognosis to prevent a stroke, and drivers an extra 2 minutes to avoid an hour of traffic once a month. Across many domains, humans do not act with simple reward functions and exhibit an aversion to severe downsides. This project analyzes how changing the reward function in reinforcement learning impacts optimal policy and outcomes. Each of the aforementioned domains has proven notoriously difficult to simulate realistically. Luckily, baseball provides a playground for exploring these ideas with discrete pitches, predictable outcomes, and public data. As such, this project seeks to learn pitch selection under different reward structures.

Every pitch in the Major League Baseball season is captured by cameras and exposed in the Statcast data set, and is the primary data source for this project. The entire project can be broken into two main steps. First a series of supervised learning models are built to estimate the distribution over pitch outcomes given a selected pitch type and location. Specifically, the models take the Statcast data as inputs and output the probability of each possible pitch outcome. This is done by a series of logistic, multinomial logistic, and ridge regressions. These models allow us to create an environment in which to simulate pitch sequences. Secondly pitch selection is formulated as a finite Markov Decision Process (MDP) with game situations as the possible states, and a series of pitch types and locations as the possible actions. The input to the system is a series of current game states and a historical dataset pitches and their outcomes. The output is then the optimal pitch selection policy maximizing the given reward function. In between, the pitch data is input into a series of supervised learning models to estimate the probability of predicted outcomes. These probabilities then feed into the MDP with randomization to drive the policy output.

2 Related Work

Prior work in baseball analytics has largely focused on prediction, or at best single decisions, not the sequential decision models I seek to explore here. Indeed, Koseler and Ceylan [5] point out that regression and classification dominate the literature. While this could be potentially helpful for the supervised learning section, they suggest SVM and knn dominate the literature and propose NN as the future. For the specific supervised problem I seek to solve, and estimate probabilistically, neither approach would yield tremendous results. Other pieces of literature, like Huang and Li [3], employ neural networks to predict entire game results. Again these studies add useful algorithms to the general field, but are more concerned with the outcome than the policy decisions. In MONEYBaRL by Sidhu and Caffo [7], they reverse the problem, modeling the batters decision to swing. However, they use a severely reduced set of actions and states, rely on MLE (effectively the smart baseline I employ) for their transition probabilities, and compute only the traditional expected value reward function. While still additive and a solid methodology, the paper does not explore the reward function space I seek to in this study.

As a natural next step should this project yield good results, Yinlam et al [2] proposed using CVaR Value iteration to create a risk sensitive MDP model. While certainly interesting, this project will only seek to replicate the result that risk sensitive objectives do alter resulting policy. Unfortunately, the approach would add complications to this project that exceed the current scope. Further to that, Bellemare et al [1] confirm that learning the overall distribution of the value (rather than just the expected value) improves performance of the algorithm. My goal with this project is to create a simulated learning environment appropriate to provide application for these theoretical results.

3 Dataset and Features

Every pitch in the MLB season is captured by the Statcast [6] data set. The data contains the entire game situation at the time of the pitch, including the runners on base, count (balls and strikes), outs, ballpark, score, teams, fielders, and player specific information. For each pitch, the speed, type, location, break, spin rate, release point, strike zone details, and pitch result are all recorded. With this, if the batter swings the swing length, bat speed, and contact details such as launch speed and angle are recorded (if there is contact). This data serves as the basis for this project and study. For this project, the data has been accessed from the pybaseball[4] API, and contains every regular season pitch of both the 2024 and the 2025 Major League Baseball (MLB) seasons.

Given the time series structure of the data, the split to training, validation, and test data for the supervised learning step has been done by date of the game. The training data set consists of games from the 2024 season and up to and including games starting July 31, 2025 in their local timezone. The validation set then contains each pitch from August 1, 2025 to August 31, 2025. Finally the test set contains each pitch from games starting on September 1, 2025 to September 30, 2025.

The raw data contains several pitches that do not conform to the pitch selection problem. Namely, the outcomes of intentional walks, automatic balls or strikes, and pitch outs are excluded from this study as the associated records do not contain the requisite data for the supervised learning portion of the study. Similarly pitches that do not contain all of the relevant feature data such as zone, release speed, or break are removed from the study.

Other features have been adjusted or derived for use as features. Some pitches have been labeled as "Pitch Out" but contain all of the relevant data. These have been collapsed into the "Other" pitch type category. The raw data contains the id for each runner. These have been collapsed into binary indicators for a runner on each base. The pitcher handedness and batter's stance have been collapsed into a singular variable indicating whether they match. The raw data provides the pitch's horizontal location from the middle of the plate and vertical distance from the ground, as well as the strike zone's top and bottom for each batter. From these, the distance from the center of the strike zone is calculated. Lastly the pitch type and the zone have been combined and encoded as one hot vectors.

The raw data also has "description" and "events" fields which contain information on pitch results and ball hit into play results respectively. From the descriptions, it can be determined if the batter swung and the result if they did or not. Specifically, the "swing" binary variable and the "no swing category" and "swing category" categorical variables are derived from the description field as in Table 6 in the Appendix. If the ball is hit into play, the events field is used to derive the "ball in play result" field. For balls in play, the data has metrics that describe the bat and ball physics of the swing and contact, and are used in the supervised learning portion with rows that contain missing values dropped. The size of the three data splits are given in Table 1 (with additional detail in the appendix Table 3). These are utilized in the supervised models, and the resulting predictions in the reinforcement learning environment.

Train	Validation	Test
1,209,873	123,876	111,086

Table 1: Total count by set

4 Methods

4.1 Supervised Learning Methodology

All of the supervised learning models share similar preprocessing steps and treatment of features, though the specific features vary slightly model to model around the same core features modeled here. The set of variables is first split into categorical variables, like pitch, zone, or their interaction, and numerical variables, like the break or speed of the pitch, count, or binary variables for base runners. Categorical variables are then one-hot encoded for use in modeling. Any unseen categories are ignored in predictive steps, as we gained no information about them in training. Numerical variables are standardized to zero mean and unit variance. These steps are applied in preprocessing for all of the supervised learning models. The specific supervised learning algorithm employed depends on the dependent variables structure. At a high level, there are three generic approaches employed in the overall model.

For binary variables, logistic regression is used to estimate the probability of the event occurring with dependent variable y modeled as $P(y|x) = \sigma(w^T x) = \frac{1}{1+e^{-w^T x}}$ with x representing the preprocessed variables used as features in the model and w a vector of the model parameters. The model estimates w by minimizing the negative log loss function with an L2 regularization term with the sklearn default value $C = 1$ applied. As this and the other loss functions are common and I only add regularization penalties, I leave the explicit loss functions to the appendix. For categorical variables with more than two classes, multinomial logistic regression (softmax regression) is employed to assign estimated probabilities to each category with y now modeled $P(y = i|x) = \phi_i(w_i^T x) = \frac{\exp(w_i^T x)}{\sum_{j=1}^k \exp(w_j^T x)}$ with w_i now a vector of the model parameters for category i . Then the model learns each of the w_i parameters similarly by minimizing the NLL with an L2 regularization term. Finally for numerical dependent variables, ridge regression is employed to yield continuous estimates. A continuous dependent variable y is modeled as $y = w^T x$ with w again a vector of the model parameters, and again estimates w by minimizing the mean squared error with an L2 regularization term. The value for lambda is automatically selecting with internal cross validation over the set $\{0.01, 0.1, 1, 10\}$ at the time of training. These models output estimates for the expected value of the variables, but each pitch yields a different result in reality. Thus I assume each y is drawn from the Gaussian distribution to introduce randomness to the environment. To estimate the standard deviation, the residual values ($r_i = y_i - \hat{y}_i$) are calculated and aggregated by pitch type and zone. Where there are more than 30 values (simple metric from CLT) in a specific aggregation, the modeled standard deviation is calculated from its set of residuals. If there are insufficient values to provide a reasonable estimate then the standard deviation of all pitches of that type is used. This standard deviation model is simplistic and addressed as a potential next step.

For both of the logistic models the default value of $C = 1$ in sklearn assigns the weight $\frac{1}{2C} = \frac{1}{2}$ to the L2 penalization. For now we leave these in place, but as will be discussed in the next steps we would ultimately look to cross validate and better tune this hyperparameter (along with comparisons to other methods). Regularization is important for all three of these approaches due to the sparse feature matrices, due to the one hot encodings, and several of the features remaining highly correlated due in the context of baseball games.

4.2 Reinforcement Learning Methodology

After the supervised learning models are trained, they will be used to take an input game situation (state) and pitch type & zone combination (action) to output a game situation (new state). This dynamic will allow for learning pitch selection using a Markov Decision Process (MDP) under different reward functions. There are 576 specific game states possible. Possible actions are defined as unique pitch type and zone combinations. The possible pitches are limited to the top 7 observed, representing a common list most pitchers throw. With 7 pitches and 13 zones, there are 91 possible actions to take. This model assumes perfect accuracy to a zone. As with all assumptions this could be relaxed in a future project as noted in the conclusion. For a complete list of possible states and actions, see Table 2.

Each episode is defined to be a half inning, starting with the bases empty, a 0-0 count (no balls, no strikes), and no outs. At the start of the inning or if the at bat ends (with an out or the batter on base), the strike zone size and batting stance vs pitching arm boolean are sampled from the training data observations to yield a realistic distribution across at bats. The half inning, and thus episode, terminates when three outs are recorded.

Given a state and action, the model uses the SL models to generate the relevant features and outcome probabilities. Specifically, the model sequentially estimates the release speed, vertical break, horizontal break, realized pitch location

Pitch Name	Count	State Variable	Eligible Values
4-Seam Fastball	385,544	balls	{0,1,2,3}
Sinker	189,389	strikes	{0,1,2}
Slider	182,325	outs_when_up	{0,1,2}
Changeup	122,335	runner1	{0,1}
Cutter	97,566	runner2	{0,1}
Sweeper	83,181	runner3	{0,1}
Curveball	78,786	stance_vs_arm_same	{0,1}

Table 2: Possible Actions (with train counts) and States

(horizontal then vertical), swing length, bat speed, launch angle, and exit velocity. Each of these is modeled as $\mathcal{N}(f_{model}(x), \sigma_{p,z}^2)$ where $\mathcal{N}$ is the normal distribution, f_{model} is the supervised model for that dependent variable, and $\sigma_{p,z}$ is the standard deviation estimates for pitch p and zone z . Then the probability of swinging, the pitch outcome, and result of any ball hit in play are estimated sequentially. A number is drawn uniformly from 0 to 1, and if less than the probability of swinging, the batter swings and if greater they do not. A similar drawing with the multinomial probability occurs to determine the outcome, and then a third to determine the result of any ball hit into play. The state is then updated under the rules of baseball assuming runners advance the same number of bases as the batter, lead runners make outs, and sacrifices advance all runners. Thus simulating each state action pair yields a new state.

The outcome of the pitch that moves the model from one state to another feeds into a reward function assigning a numeric reward based on the outcome. There are four different structures. First there is a baseline reward function that equates every 27 outs to the MLB average 4.5 runs allowed per game. There are small, linear rewards for strikes, balls, hits, etc. Secondly there is an outs only reward function where the only time a pitcher is rewarded is when they record outs. Similarly, there is a runs only function that returns a negative value whenever the pitcher allows a run. Finally there is a reward function which penalizes outcomes that lead to high scoring innings super-linearly. Penalties are scaled by base runners, multi run hits, and inversely with the number of outs. Each of these is run separately with the same simulator holding all but the reward function constant.

We nearly have everything required to begin our Markov Decision Process algorithm. The MDP is a tuple, $(S, A, \{P_{sa}\}, \gamma, R)$, where S is a set of states, A a set of actions, P_{sa} the state transition probabilities, γ the discount factor, and $R : S \times A \rightarrow \mathbb{R}$ the reward function. The states, actions, and reward frameworks are described in detail above. The probabilities, and rewards for states to action will be estimated empirically from the simulated environment. Gamma is assigned to be 0.95 since the model cares about the entire inning, so reflects the rewards from the rest of inning (up to about a 20 pitch horizon) in some capacity.

From these parameters, we define the value function for a policy, $\pi : S \rightarrow A$, according to $V^\pi(s, \pi(s)) = \mathbf{E}[R(s_0, \pi(s_0)) + \gamma R(s_1, \pi(s_1)) + \gamma^2 R(s_2, \pi(s_2))] + \dots | s_0 = s, \pi]$. The value function for policy π satisfies the Bellman equations $V^\pi(s, \pi(s)) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P_{s\pi(s)}(s' | s, \pi(s)) V^\pi(s')$. Then the optimal value function $V^*(s, a) = \max_{a \in A} [R(s, a) + \gamma \sum_{s' \in S} P_{sa}(s' | s, a) V^*(s')]$, and then the optimal policy is given $\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P_{sa}(s' | s, a) V^*(s')$.

Value iteration will be used to learn these optimal policies which starts by initializing $V(s) = 0 \forall s$. Until the values converge, set $V(s) = \max_{a \in A} [R(s, a) + \gamma \sum_{s' \in S} P_{sa}(s' | s, a) V(s')]$ This will cause V to converge to V^* . I extract the optimal actions from the final iteration.

The transition probabilities and rewards need to be estimated empirically, so the MDP is learned as follows. First, the policies are set to be uniform across all state/action pairs. While they could have been initiated with realized probabilities, this approach avoids initial biases and encourages exploration of the entire space. Then the following steps are repeated for 20 epochs. The number of epochs serves as both a reasonable amount of time for to reasonably learn the MDP, and an approximation of a reasonably long MLB pitching career.

First the optimal policy is run over 162 trials. Again this serves as both a reasonable number for a single epoch, and about the number of innings pitched for a typical starter in an MLB season (5 innings every 5 games). Thus the learned policies reasonably simulate knowledge accumulated over a 20 year MLB career. At the end of each epoch, the transition probabilities and rewards for each state and action pair are estimated by the empirical MLE, i.e., $P_{sa}(s') = \frac{\# \text{ took action } a \text{ in } s \text{ to } s'}{\# \text{ took } a \text{ in } s}$ and $R(s, a) = \frac{\text{Sum of rewards for taking } a \text{ in } s}{\# \text{ times took } a \text{ in } s}$. Then value iteration is applied with the newly updated state transition probabilities and rewards to get new estimated values, and is initialized with the prior iteration's value after the first run to speed up the process. Now if the optimal policies were updated greedily, every time the model entered an observed state it would pick the same action. There are two issues with this. First, after a single epoch there would be no more exploration in that state, so under observed actions may not get explored further even if optimal. This is an example of the classic explore vs exploit problem. Secondly, pitchers just do not do this in reality. Doing the same thing every time would create a predictable pattern batters can pick up on. So to both better adjust for the explore vs exploit tradeoff and present a more realistic model, this MDP learns through conservative policy iteration. As such, the new policy is updated per the equation $\pi_{new}(a|s) = (1 - \alpha)\pi_{old}(a|s) + \alpha \mathbf{1}\{a = a^*(s)\}$ where $a^*(s)$ is the newly calculated optimal action. The step size is set to 0.2, $\alpha = 0.2$. This is conservative, and encourages exploration throughout the entire action state from one iteration to the next.

The final output of this process is a policy that maps every game situation to a probability distribution over the pitch type and zone combination. Convergence is not expected given the constant value of alpha, so it is clear the learned MDP will have probabilistic optimal policies mimicking real pitcher behavior.

5 Experiments, Results, and Discussion

5.1 Supervised Learning Experiments

All supervised learning models are implemented using the scikit-learn library in Python under the methodology described above. To determine model accuracy and performance, I compare each model to two separate baselines. First I define a global baseline that simply outputs the average probability or value from the training set for all inputs. Given the quality of MLB hitters, This is overly simplistic as their probability of swinging at a pitch down the middle is vastly different than one wide of the plate. As such, I calculate a "smart" baseline that similarly takes an average but only of that pitch type and zone combination, using the global baseline as a fallback.

To measure performance for the logistic regression models of categorization dependent variables, I used log-loss and area under the precision-recall curve (AUPRC). The log-loss rewards high conviction correct answers and heavily penalizes high conviction wrong answers. Since the RL model relies on accurate probabilities, I want to ensure the supervised models are more both accurate and higher conviction than just guessing the overall or aggregated average. However some baseball events are rarer than others, and the majority classes could dominate and disguise inaccuracies of rarer classes if I only looked at this metric. So by using AUPRC, which is more

robust to imbalanced datasets, I can ensure that this imbalance does not meaningfully occur relative to the baselines. Further for multiple classes, I can use AUPRC on each class to see how the models performs for each. To measure performance for the linear regression models continuous numerical variables, I used mean squared error (MSE) and r^2 . The MSE provides a direct measurement of the model's prediction error, and penalizes large errors super-linearly which again confirms more accurate, higher conviction estimates than just the baselines. The r^2 metric provides information of how well the features explain variation, and can provide useful information about overfitting across the train and validation sets.

These performance metrics are calculated and compared to the two baselines for both the training and the validation datasets. Once the models were properly calibrated, a final test was performed using the test data set. Each of the probability models ran with the test metric, and were combined to estimate the probability of each outcome. These were compared using log-loss, AUPRC, and AUPRC by class against the baselines in this previously unobserved test dataset.

5.2 Supervised Learning Results and Discussion

The evaluation metrics for the categorical variables is given in Table 3 with AUPRC by class in Table 7 in the Appendix and for the numerical variables in Table 4. The end to end estimation of probabilities test has evaluation metrics compared to the baseline shown in Table 5 with the by class AUPRC available in Table 8 in the Appendix.

Across all of the categorical models, the logistic models consistently outperform both the global and the smart baselines in both AUPRC and log-loss. This indicates the models are both more accurate and better calibrated than the frequency baselines. Train and validation metrics are close indicating limited overfitting. The Supervised models outperform both baselines in each class as measured by AUPRC by specific class, indicating good performance even with the rarer cases. Here the validation measures do underperform, but given the small samples in the validation set more observations need to be observed before I can comment on overfitting in those classes. Given the good results and focus on the supervised learning portion, I did not tune the hyperparameters beyond their default settings. This will be left as a next step along with different modeling approaches, features, and additional datapoints.

The numerical ridge regression models tell a similar story, outperforming both baselines with lower MSE and higher r^2 across each model. While better than the smart baseline (and trivially better than the global baseline), the r^2 values for the pitch dynamics aside from horizontal break are significantly higher than those of the batting mechanics and ball contact. This makes sense given the small differences stemming from slight changes in the batter's swing, but could perhaps be improved with additional features or other modeling approaches. The train and validation scores are again similar, suggesting the cross validated ridge penalty helps reduce overfitting despite a sparse, high dimensional feature set, which encouraged testing different weight penalties in the first place. Given the results, the hyperparameters for this particular model were deemed acceptable, and no further tuning was required. Again improved hyperparameter exploration will be left as a next step along with different modeling approaches, features, and additional datapoints.

On the fully held out test set, the Supervised Model probabilities again outperform both baselines by a large margin over the entire model and in all but one of the individual classes (outs where the smart baseline barely outperforms it). This indicates that the full set of probability models generalizes well to unseen data, and thus is suitable for utilization in the simulation environment.

5.3 Reinforcement Learning Experiments

At the end of the 20 epochs of 162 half innings each, the final optimal policy values are stored for each of the four reward functions. Each of these four policies are then run through the MDP for 900 half innings with information stored for every pitch and half inning episode. Each of the outputs are then compared against both each other and the 2025 MLB season across several metrics. The number runs, strikeouts, hits, home runs, and walks per nine innings provide insight into how each of the reward functions shapes the MDP in comparison to the other models and reality. Additionally, I will look at the pitch type and location probabilities in each of them models to determine how differently they each learned to pitch. This is more instructive than simply comparing the probability metrics since the probabilities of observing every state varies. Additionally, I will compare the number of innings with more than 3 runs in order to see if the risk aversion worked well.

5.4 Reinforcement Learning Results and Discussion

The results from the Reinforcement Learning experiments are given in Figure 1 with the MLB data coming from Baseball Reference [8]. Each of the four reward functions clearly lead to different optimal pitch selection as shown in the pitch distributions and heatmaps.

Metric	Dataset	Model	Swing Decision	No Swing Outcome	Swing Outcome	Hit Into Play
AUPRC	Validate	Supervised	0.8168	0.6878	0.5036	0.2255
		Smart Baseline	0.7124	0.6292	0.4616	0.1109
		Baseline	0.4765	0.3333	0.3333	0.1000
AUPRC	Train	Supervised	0.8181	0.6886	0.5014	0.2324
		Smart Baseline	0.7160	0.6274	0.4625	0.1128
		Baseline	0.4788	0.3333	0.3333	0.1000
Log Loss	Validate	Supervised	0.5015	0.1873	0.9870	0.9425
		Smart Baseline	0.5904	0.2641	1.0164	1.1673
		Baseline	0.6921	0.6526	1.0842	1.1520
Log Loss	Train	Supervised	0.4997	0.1867	0.9879	0.9319
		Smart Baseline	0.5891	0.2685	1.0150	1.1310
		Baseline	0.6922	0.6530	1.0845	1.1455

Table 3: Evaluation metrics categorical variables

Metric	Dataset	Model	Release Speed	Vertical Bk	Horizontal Bk	Plate X	Plate Z	Swing Length	Bat Speed	Launch Angle	Launch Speed
MSE	Validate	Supervised	9.8108	0.1423	0.6196	0.1471	0.2161	0.4962	26.9518	704.7821	154.1273
		Smart Baseline	9.9554	0.2386	0.7189	0.1477	0.2288	0.5392	47.8596	759.1833	190.2116
		Baseline	39.0400	1.2433	0.8013	0.6744	0.9407	0.6862	50.9984	826.5724	219.3208
MSE	Train	Supervised	8.5428	0.1260	0.6294	0.1509	0.2118	0.4753	26.7228	703.3316	149.4837
		Smart Baseline	8.6679	0.1960	0.7204	0.1517	0.2245	0.5204	47.2168	766.1651	184.6943
		Baseline	36.4724	1.1509	0.8104	0.6936	0.9286	0.6611	50.4478	830.6998	211.7965
r^2	Validate	Supervised	75%	89%	23%	78%	77%	28%	47%	15%	30%
		Smart Baseline	74%	81%	10%	78%	76%	21%	6%	8%	13%
		Baseline	0%	0%	0%	0%	0%	0%	0%	0%	0%
r^2	Train	Supervised	77%	89%	22%	78%	77%	28%	47%	15%	29%
		Smart Baseline	76%	83%	11%	78%	76%	21%	6%	8%	13%
		Baseline	0%	0%	0%	0%	0%	0%	0%	0%	0%

Table 4: Evaluation metrics numerical variables

Metric	Model	Outcome
AUPRC	Supervised	0.2041
	Smart Baseline	0.1302
	Baseline	0.0714
Log Loss	Supervised	1.0969
	Smart Baseline	1.2517
	Baseline	1.5367

Table 5: Evaluation metrics against realized test set

In comparison to the MLB season statistics, the learned models throw significantly fewer balls and there are not as many walks as a result. This is likely attributable to the perfect accuracy as pitchers never "miss" their intended spot. However, the number of strikes and strikeouts are quite similar. In each of the learned models, there are more hits and runs than in reality. Only with the Limit Big Innings reward function does the learned model produce less runs per 9 innings, but each is reasonably close to the MLB. Other than the noted items, the MLB numbers do not appear dissimilar to the learned models. This is encouraging, and with improved modeling, better assumptions, and other enhancements future projects could close the gaps to better simulate reality.

For the time being, I created a reasonable playground to test the different reward functions. Each of the reward functions seemed to optimize along the intended path. The outs only model selected less fastballs than other approaches, and chose to target the heart of the plate. The strategy seems to try and throw pitches that would maximize outs for balls hit in play. The location encourages more swings, more balls in play, and quicker plate appearances (it has the least pitches per plate appearance). The runs only reward function encourages much more aggressive pitching. Clearly the model has learned to throw fastballs at the corners (particularly down and in) and outside the zone trying to get batters to chase. This yields a lot more walks, but the least hits, doubles, and home runs of the modeled environments. It takes a lot more pitches to accomplish this, and thus longer to generate outs. The baseline reward function yields results right in between these two, though much closer to the outs only variety of pitches than the runs only approach of almost all fast balls. Finally the limit big innings approach seems to accomplish its goal, with significantly fewer innings allowing three runs or more. In fact, this approach leads to the fewest runs. While the model seemed to pump the strike zone like the outs only approach, the model varied its pitches and stayed away from the center of the plate.

The hyperparameters for these models certainly need to be explored further. The choice for alpha and gamma were relatively arbitrary, with the goal of analyzing the impact of different reward functions taking precedence. Additionally, the number of epochs and the number of episodes per epoch were based on generic rules of thumb, and similarity to a human's career. The advantage of the simulated environment is to learn from far more iterations than humans could attain through nature.

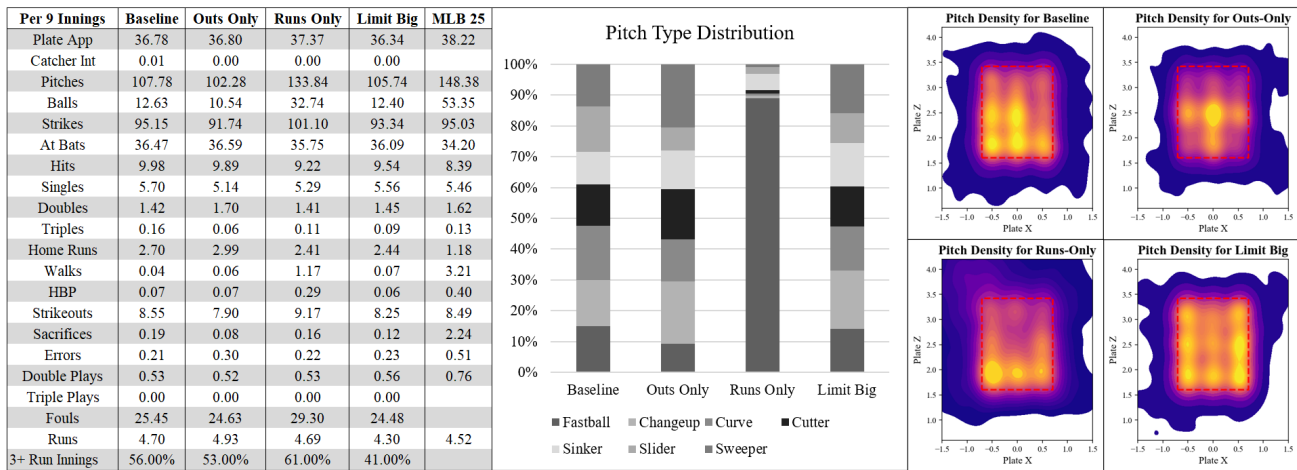


Figure 1: RL Experiment Results: Per 9 Inning Stats, Pitch Distribution, and Pitch Location Heatmaps for each of the 4 reward functions

6 Conclusion and Next Steps

This project analyzed the impact of non-linear reward functions on optimal policy generation within a simulated baseball environment. To do this, the algorithm learned a Markov Decision Process (MDP) for pitch selection. The environment was simulated using Supervised Learning algorithms which offered enhanced accuracy from baseline measures and provided the transition dynamics. The Supervised Learning models demonstrated this performance across classification and regression metrics, and successfully generalized to a held out test set. The Reinforcement Learning experiments demonstrated that the optimal pitching policy is highly sensitive to the underlying reward structure.

This study covered a broad scope of techniques and steps, and each piece therefore serves as a foundation for future study. However, these foundational pieces each can be improved and iterated upon. First the model assumed perfect accuracy, Gaussian distributions of numerical variables, and other not necessarily realistic simplifying assumptions. Each of these can be studied and either relaxed or made more realistic. Secondly only one type of model was tested for each SL and RL task. Particularly with the nature of baseball statistics, algorithms like XGBoost and/or a Random Forest regression could increase accuracy and predictive power over the logistic and linear regressions currently employed. Similarly, Q learning or other more advanced RL techniques could improve upon the MDP approach here. With changes in modeling, the hyperparameter space needs to be explored and studied in order to properly tune the model of choice. Finally, the model relied on a single data set and limited computing power. By expanding both historically and across data sets to add features, the supervised learning portion can gain predictive power (particularly for the batting physics). Similarly, running the MDP over more epochs and episodes per epoch could allow for better exploration of the action/state space. Ultimately decreasing alpha as the epoch increases, or otherwise tending towards exploiting after sufficient exploration, could lead to better optimal policy.

In practice the simple L2-regularized logistic and linear models were sufficient to create a useful environment, though batting physics could be improved with better feature selection, and the conservative policy iteration captured reward-driven strategic differences. Overall the project serves as a viable foundation for future study, and proposes a general framework to better understand how to model optimal policy under different reward structures.

Appendices

Loss Functions

- Binary Logistic: $\min_w - \left[\sum_{i=1}^n y^{(i)} \log \left(\frac{1}{1 + \exp(-w^T x^{(i)})} \right) + (1 - y^{(i)}) \log \left(1 - \frac{1}{1 + \exp(-w^T x^{(i)})} \right) \right] + \frac{1}{2} \|w\|_2^2$
- Multinomial Logistic: $\min_w - \left[\sum_{i=1}^n \log \prod_{l=1}^k \left(\frac{\exp(w_l^T x^{(i)})}{\sum_{j=1}^k \exp(w_j^T x^{(i)})} \right)^{1\{y^{(i)}=l\}} \right] + \frac{1}{2} \sum_j^k \|w_j\|_2^2$
- Ridge Regression: $\min_w \frac{1}{n} \sum_{i=1}^n \|y - w^T x\|_2^2 + \lambda \|w\|_2^2$

Tables and Figures

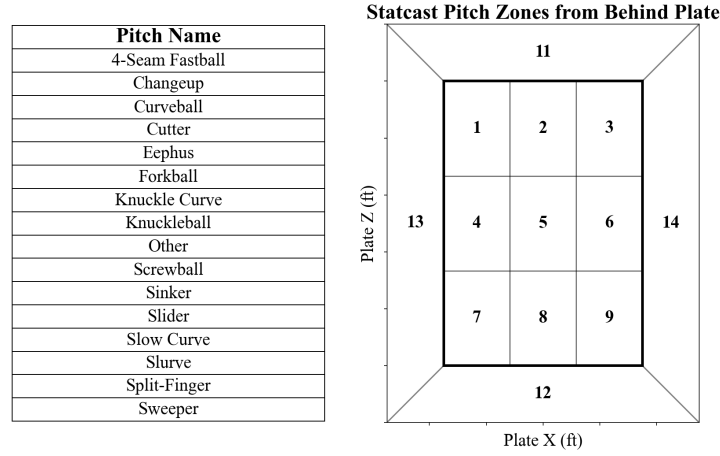


Figure 2: Statcast Pitch Names and Zones

Category	Original Description	Mapped Description
No Swing	hit by_pitch	hit by_pitch
	called_strike	called_strike
	ball	ball
	blocked_ball	
Swing	foul	foul
	foul_pitchout	
	foul_tip	
	swinging_strike	strike
	swinging_strike_blocked	
	hit_into_play	hit_into_play
Ball in Play	home_run	home_run
	triple	triple
	double	double
	single	single
	double_play	double_play
	grounded_into_double_play	
	sac_fly_double_play	
	triple_play	triple_play
	field_error	field_error
	catcher_interference	catcher_interference
	sac_but	sacrifice
	sac_fly	
	field_out	out
	fielders_choice	
	fielders_choice_out	
	force_out	

Table 6: Description Mappings to Outcomes in pre-processing

Total Feature Count in Data Sets			
Feature	Train	Validation	Test
api_break_x_batter_in	1,209,873	123,876	111,086
api_break_z_with_gravity	1,209,873	123,876	111,086
balls	1,209,873	123,876	111,086
bat_speed	546,975	57,361	51,857
launch_angle	404,491	41,207	36,482
launch_speed	403,906	41,140	36,423
outs_when_up	1,209,873	123,876	111,086
pitch_name	1,209,873	123,876	111,086
pitch_vs_sz_mid	1,209,873	123,876	111,086
pitch_zone	1,209,873	123,876	111,086
plate_x	1,209,873	123,876	111,086
plate_z	1,209,873	123,876	111,086
release_speed	1,209,873	123,876	111,086
runner1	1,209,873	123,876	111,086
runner2	1,209,873	123,876	111,086
runner3	1,209,873	123,876	111,086
stance_vs_arm_same	1,209,873	123,876	111,086
strikes	1,209,873	123,876	111,086
swing_length	546,975	57,361	51,857
sz_bot	1,209,873	123,876	111,086
sz_top	1,209,873	123,876	111,086
zone	1,209,873	123,876	111,086

(a) Observations of each feature used in Supervised Learning

Count by Pitch Type			
pitch_name	Train	Validation	Test
4-Seam Fastball	385544	39719	36570
Changeup	122335	13173	12243
Curveball	78786	9191	8566
Cutter	97566	8826	7488
Eephus	1284	155	126
Forkball	554	132	0
Knuckle Curve	20951	2160	1762
Knuckleball	1190	6	3
Other	1211	279	36
Screwball	181	0	3
Sinker	189389	19251	16686
Slider	182325	18041	16039
Slow Curve	376	47	2
Slurve	6006	436	432
Split-Finger	38994	3853	3525
Sweeper	83181	8607	7605

(b) Observations of each feature used in Supervised Learning

Count by Zone			
zone	Train	Validation	Test
1	52169	5395	4822
2	62976	6533	6034
3	47032	4692	4274
4	72558	7772	6690
5	92145	9589	8560
6	74176	7581	6914
7	57798	6125	5303
8	77980	8270	7239
9	68184	7081	6138
11	131452	13140	12165
12	101717	9946	9492
13	148581	15451	13905
14	223105	22301	19550

(c) Observations of each feature used in Supervised Learning

Figure 3: Detailed observatoin count breakdown

AUPRC by Class		Hit Into Play Outcome										No Swing Outcome			Swing Outcome			
Dataset	Model	Catch	Int	Double	DP	Error	HR	Out	Sac	Single	Triple	TP	Ball	HBP	Strike	Foul	Hit in Play	Strike
Validate	Supervised	0.001		0.136	0.239	0.021	0.445	0.773	0.350	0.267	0.015	0.009	0.986	0.114	0.963	0.443	0.497	0.570
	Smart Baseline	0.000		0.073	0.042	0.010	0.076	0.658	0.016	0.228	0.005	0.001	0.957	0.024	0.907	0.437	0.482	0.466
	Baseline	0.000		0.061	0.027	0.008	0.049	0.632	0.012	0.206	0.005	0.000	0.682	0.005	0.313	0.377	0.368	0.255
Train	Supervised	0.007		0.152	0.259	0.024	0.437	0.774	0.347	0.266	0.019	0.038	0.986	0.117	0.962	0.441	0.496	0.567
	Smart Baseline	0.003		0.076	0.043	0.013	0.066	0.667	0.018	0.231	0.009	0.000	0.953	0.023	0.906	0.435	0.483	0.470
	Baseline	0.000		0.061	0.028	0.008	0.044	0.635	0.012	0.205	0.005	0.000	0.681	0.005	0.313	0.377	0.367	0.256

Table 7: AUPRC by class, SL process with realied test set

	Ball	Hit by Pitch	Foul	Catcher's Interference	Double	Double Play	Error	Home Run	Out	Sacrifice	Single	Triple	Triple Play	Strike
Supervised	0.8699	0.1113	0.3559	0.000037	0.1017	0.1494	0.0106	0.3602	0.1842	0.0138	0.1439	0.0083	0	0.5476
Smart Baseline	0.7323	0.0145	0.3053	0.000018	0.0219	0.0096	0.0033	0.0197	0.1910	0.0043	0.0640	0.0014	0	0.4557
Baseline	0.3592	0.0026	0.1815	0.000018	0.0098	0.0047	0.0014	0.0076	0.1036	0.0021	0.0334	0.0007	0	0.2934

Table 8: AUPRC by class, SL process with realied test set

Contributions

Patrick Hartnett is the sole contributor to the project, and utilized ChatGPT as he would a student collaborator.

References

- [1] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, 2017.
- [2] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone. Risk-sensitive and robust decision-making: a cvar optimization approach. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. 2015.
- [3] M. Huang, D. Chen, P. Liu, T. Li, and R. Lai. Use of machine learning and deep learning to predict the outcomes of major league baseball matches. *Applied Sciences*, 11(12):5673, 2021.
- [4] James LeDoux, Moshe Schorr, Public Contributors. pybaseball: A python package for baseball data analysis, 2023 (version, package originally published 2017). Version 2.2.7.
- [5] K. Koseler and M. Ceylan. Machine learning applications in baseball: A systematic literature review. *Applied Artificial Intelligence*, pages 745–763, 2017.
- [6] Major League Baseball Advanced Media (MLBAM). Statcast Data: All Gameday Pitches and Events, 2015–Present. Accessed via the pybaseball API.

- [7] G. Sidhu, T. Gurney, and C. Goutte. Moneybarl: Exploiting pitcher decision-making using reinforcement learning. *The Annals of Applied Statistics*, 8(2):926 – 955, 2014.
- [8] Sports Reference LLC. Baseball-Reference.com: Baseball statistics and history, Accessed [Current Year]. Accessed [Current Date].